

ELA -1 FOR AIML

Title: **Text-to-Speech Bot**

Name	Roll No
V.Vamshi Krishna	22071A04T4
Mohammed Roshan	23075A0425

Introduction

A Text-to-Speech (TTS) bot is an application that converts written text into spoken words using speech synthesis technology. This bot allows users to enter text, select a voice, adjust the speech rate, and listen to the generated speech. The main objective is to enhance accessibility, enable assistive technology, and create interactive applications with voice output.

Objectives

1. Implement a user-friendly TTS bot using web technologies.
2. Provide users with multiple voice options for customization.
3. Allow speech rate adjustments for better user control.
4. Ensure compatibility across different browsers.
5. Develop an intuitive user interface for easy interaction.

Implementation Steps

1. Define the Purpose

- The bot will convert text to speech using the Web Speech API.
- Users can select different voices and control the speed of speech.

2. Choose Development Platform and Technology Stack

- **Language:** JavaScript (React.js)
- **Browser API:** Web Speech API
- **User Interface:** HTML, CSS, React Components

3. Develop the Core Functionality

- Fetch available voices using `window.speechSynthesis.getVoices()`.
- Implement user controls for voice selection and speech rate.
- Integrate text input and buttons for speaking and stopping speech.

4. Design and Implement UI

- A text area for input.
- A dropdown menu to select a voice.
- A slider to adjust speech rate.
- Buttons to start and stop speech.

5. Testing and Debugging

- Ensure smooth voice selection and playback.
- Test across different browsers.
- Handle errors such as empty input and unsupported browsers.

6. Deployment and Maintenance

- Deploy as a web application.
- Continuously update for new browser features and voice improvements.

Key Design Procedures

1. Speech Synthesis Integration:

- o Uses `SpeechSynthesisUtterance` to convert text to speech.
- o Allows dynamic voice selection and speech rate adjustment.

2. User Experience Enhancements:

- o Provides real-time feedback for selected voices and rate changes.
- o Includes alert messages for invalid inputs.

3. Cross-Browser Compatibility:

- o Ensures support for major browsers with Web Speech API.

4. Performance Optimization:

- o Cancels ongoing speech before starting a new one.
- o Efficiently loads voice options on initialization.

React Implementation Code

```
import React, { useState, useEffect } from "react";
```

```
function App() {
  const [text, setText] = useState("");
  const [voices, setVoices] = useState([]);
  const [selectedVoice, setSelectedVoice] = useState(null);
  const [rate, setRate] = useState(1);

  useEffect(() => {
    const loadVoices = () => {
      const availableVoices = window.speechSynthesis.getVoices();
      setVoices(availableVoices);
      if (availableVoices.length > 0) {
        setSelectedVoice(availableVoices[0].name);
      }
    };
    loadVoices();
    window.speechSynthesis.onvoiceschanged = loadVoices;
  }, []);

  const speakText = () => {
    if (!text.trim()) {
      alert("Please enter text to speak!");
      return;
    }
  }
}
```

```

    if ("speechSynthesis" in window) {
      window.speechSynthesis.cancel();
      let speech = new SpeechSynthesisUtterance(text);
      speech.rate = rate;
      const voice = voices.find((v) => v.name === selectedVoice);
      if (voice) speech.voice = voice;
      window.speechSynthesis.speak(speech);
    } else {
      alert("Text-to-Speech is not supported in this browser.");
    }
  };

  const stopSpeaking = () => {
    window.speechSynthesis.cancel();
  };

  return (
    <div style={{ textAlign: "center", marginTop: "50px" }}>
      <h2>Welcome to our Text-to-Speech Bot</h2>
      <textarea rows="4" cols="50" placeholder="Enter text..." value={text} onChange={(e) =>
setText(e.target.value)}></textarea>
      <br />
      <label>Choose Voice: </label>
      <select onChange={(e) => setSelectedVoice(e.target.value)} value={selectedVoice}>
        {voices.map((voice, index) => (
          <option key={index} value={voice.name}>
            {voice.name} ({voice.lang})
          </option>
        ))}
      </select>
      <br />
      <label>Speech Rate: {rate}</label>
      <input type="range" min="0.5" max="5" step="0.5" value={rate} onChange={(e) =>
setRate(e.target.value)} />
      <br />
      <button onClick={speakText} style={{ marginTop: "10px", padding: "10px", marginRight: "10px"
}}>
        Speak
      </button>
      <button onClick={stopSpeaking} style={{ marginTop: "10px", padding: "10px", backgroundColor:
"red", color: "white" }}>
        Stop
      </button>
    </div>
  );
}

export default App;

```

Final bot result:

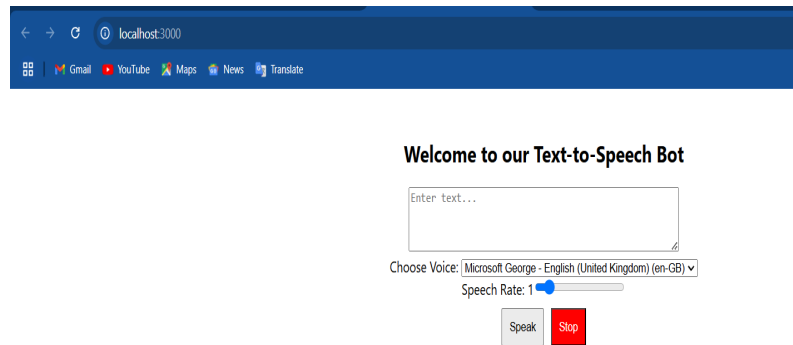


Fig1.1:before user gives input

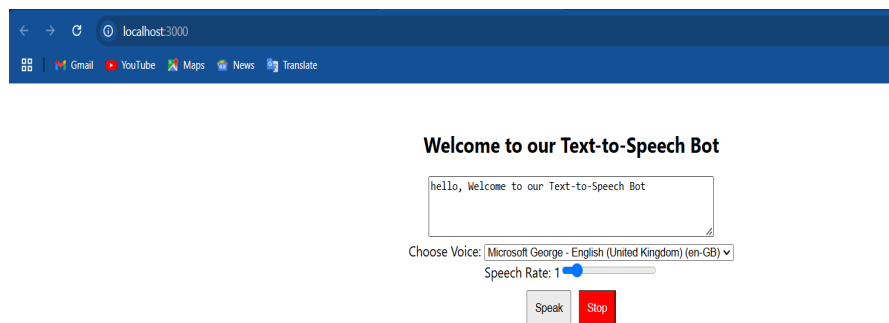


Fig 1.2:after user gives input

Conclusion

The Text-to-Speech Bot provides an interactive and customizable speech synthesis experience. By utilizing the Web Speech API within a React-based UI, users can easily convert text into speech with various voices and adjustable speeds. The bot ensures accessibility, ease of use, and smooth performance across different browsers.